

Tiny LLM Demo — Guida al codice

Struttura, sintassi Python e logica del progetto, per chi viene da altri linguaggi OOP

Indice

Introduzione

Parte 1 — Sintassi Python essenziale

- 1.1 Niente parentesi graffe: l'indentazione è sintassi
- 1.2 Definire funzioni: def
- 1.3 Type hints: tipizzazione facoltativa, non forzata
- 1.4 Classi: class, self, __init__
- 1.5 List comprehension: un for compattato in un'espressione
- 1.6 Tuple e "multiple return values"
- 1.7 Argomenti con valore di default (keyword arguments)
- 1.8 f-string: interpolazione di stringhe
- 1.9 Import e moduli
- 1.10 if __name__ == "__main__":
- 1.11 Context manager: with ... :
- 1.12 Docstring: """ ... """
- 1.13 Eccezioni: raise, try / except
- 1.14 Insiemi: set e l'operatore |
- 1.15 Path invece di stringhe per i percorsi

Parte 2 — Struttura del progetto

Parte 3 — Walkthrough file per file

- 3.1 data.py — preparazione dei dati
- 3.2 model.py — l'architettura della rete
- 3.3 train.py — il ciclo di training
- 3.4 generate.py — inferenza
- 3.5 demo.py — orchestrazione interattiva
- 3.6 tests/test_smoke.py — verifica automatica

Cose da tenere a mente, in sintesi

Riferimenti

Introduzione

Questo documento spiega il progetto `tiny-llm-demo` riga per riga, partendo dal presupposto che tu **non conosca Python** come linguaggio ma abbia esperienza solida con OOP (classi, interfacce, ereditarietà) in altri linguaggi (Java, C#, TypeScript, ecc.).

L'obiettivo non è insegnarti Python in generale, ma darti gli strumenti per leggere *questo specifico progetto* senza sentirti perso sulla sintassi, e capire perché è strutturato così.

Il documento è diviso in tre parti:

1. **Sintassi Python essenziale** — le cose che ti servono per leggere il codice, spiegate per confronto con OOP classico.
2. **Struttura del progetto** — come i file si parlano tra loro.

3. Walkthrough file per file — cosa fa ogni riga di codice importante.

Il complemento concettuale di questa guida (cosa succede *dentro* la rete, come impara, come genera) è nel documento *Principi della rete neurale*.

Parte 1 — Sintassi Python essenziale

1.1 Niente parentesi graffe: l'indentazione è sintassi

In Java/C# i blocchi di codice sono delimitati da `{ }`. In Python, il blocco è delimitato **dall'indentazione stessa** — non è una convenzione di stile, è sintassi obbligatoria.

```
def somma(a, b):
    risultato = a + b      # questa riga fa parte della funzione (indentata)
    return risultato       # anche questa

print("fuori dalla funzione") # non indentata: fuori dal blocco
```

Se sbagli l'indentazione, il codice non compila (in Python si dice: solleva un `IndentationError`). Non ci sono `;` a fine riga né `{ }`.

1.2 Definire funzioni: `def`

Equivalente di un metodo statico/una funzione standalone in altri linguaggi:

```
def build_vocab(corpus):
    ...
```

`def` = "sto definendo una funzione". Non c'è un tipo di ritorno dichiarato obbligatoriamente (vedi 1.3 sui type hints), non c'è overload di funzioni (due funzioni con lo stesso nome, anche con firme diverse, semplicemente si sovrascrivono a vicenda).

1.3 Type hints: tipizzazione facoltativa, non forzata

Python è dinamicamente tipizzato: normalmente non dichiarare i tipi. Ma *può* avere delle annotazioni di tipo (chiamate "type hints") — sono solo documentazione/aiuto per l'IDE, **non vengono controllate a runtime** come farebbe il compilatore Java/C#.

```
def build_vocab(corpus: List[str]) -> Tuple[Dict[str, int], Dict[int, str]]:
```

Si legge: "riceve `corpus` di tipo `List[str]` (lista di stringhe), ritorna una `Tuple` (coppia) di due `Dict` (dizionari)". `List`, `Dict`, `Tuple` vengono importati da un modulo chiamato `typing`. Sono l'equivalente concettuale di `List<String>`, `Map<K,V>` in Java, ma puramente informativi: se passi il tipo sbagliato, Python non si lamenta finché non prova a usarlo in modo incompatibile a runtime.

Nel progetto compaiono anche annotazioni su variabili di modulo (`CORPUS: List[str] = [...]`, `END: str = "<end>"`, `TEMPERATURE_DEFAULT: float = 0.6`): stesso principio, servono solo a chi legge.

1.4 Classi: `class`, `self`, `__init__`

Questo è il punto di maggiore differenza rispetto a OOP "classico". Confronto diretto:

```
// Java
public class TinyLM {
    private Embedding embedding;

    public TinyLM(int vocabSize, int embeddingDim) {
        this.embedding = new Embedding(vocabSize, embeddingDim);
    }
}
```

```
# Python
class TinyLM(nn.Module):
    def __init__(self, vocab_size, embedding_dim):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
```

Punti chiave:

- `class TinyLM(nn.Module):` — TinyLM **eredita da** `nn.Module` (come `extends` in Java). `nn.Module` è la classe base di PyTorch per "qualunque cosa sia un pezzo di rete neurale".
- `__init__` è il **costruttore** (equivalente di un costruttore Java/C# con lo stesso nome della classe). I doppi underscore (`__init__`) sono una convenzione Python per metodi "speciali"/gestiti dal linguaggio — se li vedi, pensa "hook del linguaggio", non un tuo metodo.
- `self` è **esplicito**: ogni metodo di istanza riceve `self` come primo parametro, ed è l'equivalente di `this`. In Java `this` è implicito; in Python devi scriverlo sempre a mano, sia nella firma del metodo che quando accedi ai campi (`self.embedding`, non `embedding`).
- `super().__init__()` — chiama il costruttore della classe padre, come `super()` in Java. **È obbligatorio farlo per primo** quando erediti da `nn.Module`, altrimenti PyTorch non riesce a tracciare i parametri della rete (vedi Parte 3).
- Non esistono modificatori di accesso (`private` / `public` / `protected`). Tutto è pubblico per default. La convenzione per "questo è privato, non toccarlo da fuori" è un underscore iniziale (`_qualcosa`), ma è solo una convenzione, nessuno la impone.

1.5 List comprehension: un `for` compattato in un'espressione

Una delle sintassi più caratteristiche di Python, che non ha un vero equivalente diretto in Java/C# (il più vicino sono gli Stream di Java o LINQ in C#).

```
parole_generate = [itos[i] for i in ids]
```

Si legge da sinistra: "crea una lista contenente `itos[i]`, per ogni `i` in `ids`". Equivale a:

```
// Java, per confronto concettuale
List<String> paroleGenerate = new ArrayList<>();
for (int i : ids) {
    paroleGenerate.add(itos.get(i));
}
```

Stessa idea per i **dizionari** (l'equivalente Python di `Map` / `HashMap`):

```
stoi = {parola: i for i, parola in enumerate(parole)}
```

"Crea un dizionario dove la chiave è `parola` e il valore è `i`, per ogni coppia `(i, parola)` prodotta da `enumerate(parole)`". `enumerate()` è una funzione builtin che, data una lista, ti restituisce coppie (indice, elemento) — utile per non dover gestire manualmente un contatore come faresti con un `for (int i=0; ...)`.

Una comprehension può essere anche un **generatore** passato direttamente a una funzione, senza creare la lista intermedia:

```
sorted(p for p in stoi if p != END)
```

"Ordina tutte le `p` prese da `stoi`, ma solo quelle diverse da `END`". Il `for ... if ...` finale è il filtro, come `.filter(...)` in uno Stream.

1.6 Tuple e "multiple return values"

Python permette di ritornare più valori da una funzione senza creare una classe/record ad hoc:

```
def build_vocab(corpus):  
    ...  
    return stoi, itos    # ritorna DUE valori
```

Chi chiama la funzione può "spacchettare" (unpacking) direttamente:

```
stoi, itos = build_vocab(corpus)
```

È concettualmente vicino a un `Tuple<A,B>` in C#, ma con sintassi nativa e leggera — non serve dichiarare esplicitamente un tipo tupla.

1.7 Argomenti con valore di default (keyword arguments)

```
def train_model(corpus=CORPUS, embedding_dim=8, epochs=200, lr=0.01, verbose=True):
```

Ogni parametro ha un valore di default. Chi chiama la funzione può ometterli, oppure specificarli per nome, in qualsiasi ordine:

```
train_model(epochs=500)           # usa i default per tutto il resto  
train_model(CORPUS, embedding_dim=16) # posizionale + per nome  
train_model(corpus_corrente, verbose=False)
```

Non esiste overload di metodi in Python: questo meccanismo (default + keyword arguments) è il modo idiomático per ottenere flessibilità simile a "più costruttori/metodi overloaded".

1.8 f-string: interpolazione di stringhe

```
print(f"epoch {epoch:4d} | loss {loss.item():.4f}")
```

Il prefisso `f` prima delle virgolette rende la stringa "formattata": tutto quello dentro `{ }` viene valutato come espressione Python e inserito nella stringa. `:4d` e `:.4f` sono specificatori di formato (allinea su 4 cifre, mostra 4 decimali) — concettualmente equivalente a `String.format` in Java o all'interpolazione `${variabile}` in C#.

1.9 Import e moduli

```
from .data import CORPUS, END, build_vocab, build_dataset
from .model import TinyLM
```

Il punto (`.`) prima di `data` indica un **import relativo**: "prendi il modulo `data.py` nella stessa cartella (package)". È l'equivalente di un import tra classi dello stesso namespace in C#, o dello stesso package in Java — ma qui il "modulo" è letteralmente il file `.py` stesso: in Python ogni file è automaticamente un modulo, non serve dichiarare `package` o `namespace`.

Gli import senza punto (`import re`, `import torch`, `from pathlib import Path`) prendono invece dalla libreria standard o dai pacchetti installati con `pip`.

1.10 `if __name__ == "__main__":`

Questo idiomma compare in quasi ogni file eseguibile del progetto:

```
if __name__ == "__main__":
    modello, stoi, itos, storico = train_model()
    save_checkpoint(modello, stoi, itos)
```

`__name__` è una variabile speciale che Python imposta automaticamente: vale `"__main__"` solo se il file è stato **eseguito direttamente** (es. `python -m src.train`), mentre vale il nome del modulo se il file è stato **importato** da un altro file (es. `from .train import train_model`).

Serve a distinguere "codice che deve girare solo quando lancio questo file da terminale" da "codice riutilizzabile che altri file possono importare". È l'equivalente concettuale di un metodo `Main` in C#/Java, ma un file può avere sia funzioni riutilizzabili sia un blocco `__main__` — non serve una classe `Program` dedicata.

1.11 Context manager: `with ... :`

```
with torch.no_grad():
    logits = model(x)
```

`with` apre un blocco che garantisce setup/cleanup automatico, come uno `using` in C# (`IDisposable`) o un `try/finally` implicito. `torch.no_grad()` dice a PyTorch "per la durata di questo blocco, non calcolare i gradienti" — usato in inferenza, dove non serve allenare nulla e si risparmiano calcoli inutili.

1.12 Docstring: `""" ... """`

```
def build_vocab(corpus):
    """
    Costruisce il vocabolario a partire dal corpus.
    """
```

Una stringa tra `"""` subito dopo la firma di una funzione/classe è una **docstring**: documentazione ufficiale, leggibile da IDE e strumenti (`help(build_vocab)`), concettualmente equivalente a un blocco `/** ... */` Javadoc, ma è un vero valore di stringa nel linguaggio, non un commento. In questo progetto le docstring sono lunghe e discorsive di proposito: contengono la spiegazione didattica di ogni funzione.

1.13 Eccezioni: `raise`, `try` / `except`

```
if parola_iniziale not in stoi:
    raise ValueError(f"'{parola_iniziale}' non è nel vocabolario. ...")
```

`raise` è l'equivalente di `throw`. `ValueError` è un'eccezione builtin che si usa per "argomento formalmente valido ma con un valore non accettabile" (l'analogo più vicino è `IllegalArgumentException` / `ArgumentException`). Non esiste il concetto di *checked exception*: nessuna funzione dichiara cosa può sollevare.

```
try:
    risultato = generate(modello, stoi, itos, testo, n_parole=6)
except ValueError as e:
    print(f"Errore: {e}")
```

`try` / `except` è `try` / `catch`. `except (EOFError, KeyboardInterrupt):` (in `demo.py`) cattura più tipi in un colpo solo, come un multi-catch `catch (A | B e)` in Java.

1.14 Insiemi: `set` e l'operatore `|`

```
parole = sorted(set(" ".join(corpus).split()) | {END})
```

`set(...)` crea un insieme (un `HashSet`) e `{END}` è un *set literal* con un solo elemento. L'operatore `|` fra due insiemi è l'**unione**: "tutte le parole del corpus, più il token `END`". Questa sintassi con gli operatori (`|` unione, `&` intersezione, `-` differenza) è specifica di Python; in Java/C# useresti `addAll` / `UnionWith`.

1.15 `Path` invece di stringhe per i percorsi

```
CHECKPOINT_PATH = Path(__file__).resolve().parent.parent / "checkpoint.pt"
```

`pathlib.Path` rappresenta un percorso come oggetto (come `java.nio.file.Path` o `System.IO.Path`, ma con l'operatore `/` per concatenare i pezzi). `__file__` è il percorso del file `.py` corrente; `.parent.parent` risale di due livelli (da `src/train.py` alla root del progetto). Il risultato: il checkpoint viene sempre salvato nella root, da qualunque cartella tu lanci il comando.

Parte 2 — Struttura del progetto

```
tiny-llm-demo/
├── src/
│   ├── data.py      -> dati grezzi: corpus, token END, normalizzazione, vocabolario, dataset
│   ├── model.py     -> architettura della rete (classe TinyLM)
│   ├── train.py     -> ciclo di training + salvataggio/caricamento
│   ├── generate.py  -> inferenza: genera testo dal modello allenato
│   └── demo.py      -> CLI interattiva che orchestra tutto il resto
├── tests/
│   └── test_smoke.py -> verifica automatica che la pipeline funzioni
├── docs/            -> sorgenti Markdown di questa guida e script per rigenerare i PDF
├── requirements.txt -> torch, pytest, numpy
└── checkpoint.pt    -> creato da src.train (non versionato)
```

Ogni file dipende solo da quelli "sotto" di lui, mai il contrario:

```
demo.py      --usa--> train.py, generate.py, data.py (CORPUS, normalizza_testo)
generate.py  --usa--> train.py (load_checkpoint), model.py, data.py (END)
train.py     --usa--> model.py, data.py
model.py     --usa--> (solo PyTorch, nessuna dipendenza interna)
data.py      --usa--> (nessuna dipendenza interna)
```

Questo è lo stesso principio di *dependency direction* che probabilmente applichi già in architetture a layer (controller → service → repository): i livelli "bassi" (dati, modello) non sanno nulla di come vengono usati da quelli "alti" (training, CLI).

Parte 3 — Walkthrough file per file

3.1 data.py — preparazione dei dati

Nessuna rete neurale qui: solo trasformazione testo → numeri.

```
CORPUS: List[str] = [
    "il gatto dorme sul divano",
    "il cane dorme sul tappeto",
    ...
]
```

Una semplice lista di stringhe *modulo-level* (variabile globale del modulo, visibile a chiunque importi `data.py`). Sei frasi, tutte minuscole, senza punteggiatura: 14 parole distinte in tutto.

```
END: str = "<end>"
```

Il **token di fine frase**. Non è una parola del corpus: è un simbolo in più nel vocabolario che il modello impara a predire quando la frase è finita. Senza di lui il modello non avrebbe modo di "dire basta" e continuerebbe a generare parole all'infinito. È lo stesso meccanismo dei modelli veri (il token *EOS*, end-of-sequence). Le parentesi angolari lo rendono impossibile da confondere con una parola scritta dall'utente, perché la funzione seguente toglie la punteggiatura da qualunque input.

```
def normalizza_testo(testo: str) -> str:
    return re.sub(r"^[^w\s]", "", testo.lower()).strip()
```

Usata dalla demo per ripulire le frasi inserite dall'utente. `re` è il modulo delle espressioni regolari (`java.util.regex` / `System.Text.RegularExpressions`). `r"..."` è una *raw string*: i backslash non vanno raddoppiati, quindi `\w` e `\s` arrivano alla regex così come sono. La regex `^[^w\s]` significa "qualunque carattere che *non* sia alfanumerico né spazio" — cioè la punteggiatura — e `re.sub` la sostituisce con la stringa vuota. Prima si passa tutto in minuscolo con `.lower()`, alla fine `.strip()` toglie gli spazi ai bordi. Risultato: "Il gatto, dorme!" → "il gatto dorme", quindi le parole coincidono con quelle già nel vocabolario invece di creare token "sporchi".

```
def build_vocab(corpus):
    parole = sorted(set(" ".join(corpus).split()) | {END})
    stoi = {parola: i for i, parola in enumerate(parole)}
    itos = {i: parola for parola, i in stoi.items()}
    return stoi, itos
```

Passo per passo:

1. `" ".join(corpus)` — unisce tutte le frasi della lista in un'unica stringa, separate da spazio. `join` è un metodo su stringa, chiamato con la stringa separatore come "soggetto": sintassi rovesciata rispetto a `String.join(" ", corpus)` in Java, ma stesso risultato.
2. `.split()` — spezza quella stringona in una lista di singole parole (di default divide sugli spazi bianchi).
3. `set(...)` — converte la lista in un insieme (equivalente di `HashSet`): elimina i duplicati, "gatto" compare una volta sola anche se ricorre in più frasi.
4. `| {END}` — unione con l'insieme che contiene solo il token di fine frase (vedi 1.14). Il vocabolario finale ha quindi **15 token**: 14 parole + `<end>`.
5. `sorted(...)` — riordina alfabeticamente, solo per avere un ordine deterministico e riproducibile tra run diversi (altrimenti l'ordine di un set non è garantito). `<end>` finisce per primo, con id 0, perché `<` viene prima delle lettere.
6. Le due comprehension da dizionario costruiscono le due mappe parola↔id (vedi 1.5).

```
def build_dataset(corpus, stoi):
    coppie = []
    for frase in corpus:
        ids = [stoi[parola] for parola in frase.split()] + [stoi[END]]
        for i in range(len(ids) - 1):
            coppie.append((ids[i], ids[i + 1]))

    X = torch.tensor([c[0] for c in coppie])
    Y = torch.tensor([c[1] for c in coppie])
    return X, Y
```

Qui costruiamo gli esempi di training: per ogni frase, ogni parola diventa un esempio "data questa parola, la prossima è quest'altra". Il `+ [stoi[END]]` concatena due liste (in Python `+` fra liste è la concatenazione): ogni frase viene **chiusa con il token END**, che diventa la "risposta giusta" dell'ultima parola. Esempio: "il gatto dorme" → `(il → gatto)`, `(gatto → dorme)`, `(dorme → <end>)`.

Questo serve a due cose: il modello impara a segnalare quando la frase è finita, e anche l'ultima parola di ogni frase diventa un input di training (altrimenti parole come "divano", che compaiono solo in fondo, non verrebbero mai allenate e produrrebbero output casuali). Con il corpus di default si ottengono 32 coppie.

`range(len(ids) - 1)` genera gli indici da 0 fino al penultimo elemento (l'ultimo, END, non ha una "parola dopo"). `torch.tensor(...)` converte una lista Python in un **tensore** PyTorch — la struttura dati fondamentale di PyTorch, concettualmente un array multidimensionale ottimizzato per calcolo numerico (pensa a un `float[][]` ma con superpoteri: supporta operazioni vettoriali e, più avanti, il calcolo automatico dei gradienti).

3.2 `model.py` — l'architettura della rete

```
class TinyLM(nn.Module):
    def __init__(self, vocab_size: int, embedding_dim: int = 8):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.linear = nn.Linear(embedding_dim, vocab_size)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        e = self.embedding(x)
        logits = self.linear(e)
        return logits
```

Cosa succede concettualmente:

- `TinyLM` eredita da `nn.Module`: questa è l'interfaccia/classe base che PyTorch richiede per qualunque componente di rete neurale (un layer, un intero modello, un sotto-blocco). È simile a implementare un'interfaccia tipo `IPredictable` in C#, ma con comportamento di base già fornito dalla classe padre (tracciamento automatico dei parametri, spostamento su GPU/CPU, salvataggio/caricamento pesi, ecc.).
- Nel costruttore, definiamo due sotto-moduli come attributi di istanza:
 - `nn.Embedding(vocab_size, embedding_dim)`: internamente è una matrice di numeri (`vocab_size` righe $\times$ `embedding_dim` colonne), inizializzata a caso. Dato un id, restituisce la riga corrispondente — è letteralmente una lookup table, ma i suoi valori sono **parametri allenabili** (verranno modificati dal training).
 - `nn.Linear(embedding_dim, vocab_size)`: uno strato "denso" — matematicamente una trasformazione affine (matrice + bias), anche questa con parametri allenabili.
- Il metodo `forward` è il metodo che PyTorch chiama automaticamente quando "usi" il modello come una funzione (`model(x)`). Non lo chiami mai direttamente col nome `forward`: è un hook che `nn.Module` intercetta — simile a un metodo di interfaccia che l'infrastruttura invoca per te (pensa a come un framework HTTP chiama il tuo `handleRequest` senza che tu lo invochi manualmente).

Con il corpus di default (15 token, embedding di dimensione 8) il modello ha in tutto **255 parametri**: 15×8 nell'embedding, $8 \times 15 + 15$ di bias nel layer lineare. Un LLM vero ne ha miliardi, ma la struttura "numeri allenabili organizzati in strati" è la stessa.

Perché ereditare da `nn.Module` invece di scrivere una classe qualsiasi? Perché `nn.Module` fa un lavoro invisibile ma fondamentale: tiene traccia di tutti i parametri allenabili definiti nei sotto-moduli (`self.embedding`, `self.linear`), rendendoli accessibili in blocco con `model.parameters()` — è quello che passiamo all'optimizer nel prossimo file. Senza questa "magia" dell'ereditarietà, dovresti raccogliere manualmente ogni singolo tensore di pesi sparso nel modello.

3.3 `train.py` — il ciclo di training

```
def train_model(corpus=CORPUS, embedding_dim=8, epochs=200, lr=0.01, verbose=True):
    stoi, itos = build_vocab(corpus)
    X, Y = build_dataset(corpus, stoi)

    model = TinyLM(vocab_size=len(stoi), embedding_dim=embedding_dim)
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    storico_loss = []

    for epoch in range(epochs):
        optimizer.zero_grad()
        logits = model(X)
        loss = criterion(logits, Y)
        loss.backward()
        optimizer.step()

        storico_loss.append(loss.item())
    ...
```

Questo è il cuore concettuale dell'intero progetto. Riga per riga, dentro il ciclo `for`:

1. `optimizer.zero_grad()` — PyTorch **accumula** i gradienti calcolati ad ogni `backward()`, invece di sovrascriverli. Se non li azzeri esplicitamente prima di ogni nuova iterazione, si sommerebbero a quelli

del giro precedente, corrompendo il training. È un dettaglio implementativo di PyTorch da ricordare sempre: "azzera prima di ricalcolare".

2. `logits = model(X)` — il **forward pass**: passiamo tutto il dataset `X` in un colpo solo (PyTorch vettorializza automaticamente su tutte le righe), e otteniamo un punteggio grezzo per ogni token del vocabolario, per ciascun esempio.
3. `loss = criterion(logits, Y)` — `criterion` è l'istanza di `CrossEntropyLoss` creata sopra: è essa stessa un `nn.Module`-simile, chiamabile come una funzione (stesso pattern `forward()` di prima). Confronta i punteggi predetti (`logits`) con le risposte corrette (`Y`) e produce un singolo numero: quanto il modello ha sbagliato in media su tutto il dataset.
4. `loss.backward()` — qui avviene la parte che sembra più "magica": PyTorch tiene traccia automaticamente di ogni operazione matematica eseguita nel forward pass (si chiama **autograd**, differenziazione automatica). Chiamare `.backward()` su `loss` percorre all'indietro tutto quel grafo di operazioni e calcola, per ogni singolo parametro allenabile del modello, "di quanto e in che direzione" contribuisce all'errore finale. Non è simbolico/manuale: è calcolo automatico basato su come i tensori sono stati costruiti.
5. `optimizer.step()` — applica concretamente le correzioni calcolate al punto 4 a tutti i parametri del modello (quelli restituiti da `model.parameters()`, passati all'optimizer alla creazione), scalate dal learning rate `lr`.
6. `storico_loss.append(loss.item())` — `loss` è un tensore PyTorch (anche se contiene un solo numero); `.item()` lo estrae come normale numero Python (`float`), per poterlo mettere in una lista Python normale.

Nota importante su `model(X)` vs `model.forward(X)`: nel codice chiamiamo `model(X)`, mai `model.forward(X)` direttamente. `nn.Module` implementa un metodo speciale (`__call__`, un altro degli hook con doppio underscore visti in 1.4) che fa un po' di lavoro extra prima e dopo aver chiamato `forward()`. Chiamare sempre `model(X)` e mai `.forward(X)` a mano è una convenzione PyTorch da rispettare sempre.

```
def save_checkpoint(model, stoi, itos, path=CHECKPOINT_PATH):
    torch.save(
        {
            "model_state": model.state_dict(),
            "vocab_size": len(stoi),
            "embedding_dim": model.embedding.embedding_dim,
            "stoi": stoi,
            "itos": itos,
        },
        path,
    )
```

`model.state_dict()` restituisce un dizionario `{nome_parametro: tensore_di_pesi}` — lo "snapshot" completo di tutti i numeri allenati del modello. `torch.save` lo serializza su disco insieme al vocabolario, in un unico file binario `checkpoint.pt`.

```
def load_checkpoint(path=CHECKPOINT_PATH):
    if not path.exists():
        raise FileNotFoundError(...)
    checkpoint = torch.load(path, weights_only=False)
    model = TinyLM(vocab_size=checkpoint["vocab_size"], embedding_dim=checkpoint["embedding_dim"])
    model.load_state_dict(checkpoint["model_state"])
    model.eval()
    return model, checkpoint["stoi"], checkpoint["itos"]
```

Il caricamento fa il percorso inverso: ricrea un `TinyLM` vuoto con la stessa forma, e gli inietta dentro i pesi salvati con `model.load_state_dict(...)`. Due dettagli:

- Se il file non esiste, solleviamo un `FileNotFoundError` con un messaggio che spiega cosa fare (python `-m src.train`), invece di lasciare emergere l'errore criptico di PyTorch.
- `weights_only=False`: dalla versione 2.6 di PyTorch `torch.load` di default rifiuta di caricare qualunque cosa non sia un tensore, per motivi di sicurezza (il formato `.pt` è basato su *pickle*, che può eseguire codice arbitrario). Il nostro checkpoint contiene anche i due dizionari del vocabolario, quindi dobbiamo dire esplicitamente "fidati, il file l'ho scritto io".

3.4 generate.py — inferenza

```
TEMPERATURE_DEFAULT: float = 0.6

def generate(model, stoi, itos, parola_iniziale, n_parole=5, temperature=TEMPERATURE_DEFAULT):
    if parola_iniziale not in stoi:
        raise ValueError(
            f"'{parola_iniziale}' non è nel vocabolario. "
            f"Parole disponibili: {sorted(p for p in stoi if p != END)}"
        )

    model.eval()
    parola = parola_iniziale
    frase = [parola]

    with torch.no_grad():
        for _ in range(n_parole):
            x = torch.tensor([stoi[parola]])
            logits = model(x)

            if temperature <= 0:
                prossimo_id = int(torch.argmax(logits, dim=-1).item())
            else:
                probabilita = torch.softmax(logits / temperature, dim=-1)
                prossimo_id = int(torch.multinomial(probabilita, num_samples=1).item())

            parola = itos[prossimo_id]
            if parola == END:
                break

            frase.append(parola)

    return " ".join(frase)
```

- `TEMPERATURE_DEFAULT = 0.6` — una costante di modulo, volutamente sotto 1.0: con un corpus così piccolo il campionamento "onesto" (1.0) pesca troppo spesso continuazioni rare e le frasi escono sgangherate. A 0.6 il modello segue di più i pattern che ha davvero imparato. È un parametro, non una verità: alzandolo (es. 1.5) il modello inizia a "delirare".

- Il controllo iniziale con `raise ValueError` (vedi 1.13) trasforma un `KeyError` criptico su `stoi[parola]` in un messaggio leggibile con l'elenco delle parole valide. `END` viene escluso dall'elenco: è un token interno, non ha senso come punto di partenza.
- `model.eval()` — mette il modello in "modalità inferenza". Nel nostro modello non cambia nulla di osservabile (non abbiamo layer come Dropout/BatchNorm che si comportano diversamente in training vs inferenza), ma è buona pratica chiamarlo sempre prima di generare.
- `with torch.no_grad():` — come spiegato in 1.11: qui non stiamo allenando, quindi diciamo a PyTorch di non tracciare il grafo per l'autograd, risparmiando memoria e calcoli inutili.
- `torch.softmax(logits / temperature, dim=-1)` — trasforma i punteggi grezzi in probabilità che sommano a 1 (100%). Dividere per `temperature` prima del softmax è il trucco standard per rendere la distribuzione più "piatta" (temperature alta → più casualità) o più "appuntita" (temperature bassa → quasi deterministico).
- `torch.multinomial(probabilita, num_samples=1)` — campiona casualmente un id, pesato dalle probabilità appena calcolate (non prende semplicemente il massimo). È il motivo per cui rilanciando `generate()` più volte ottieni frasi leggermente diverse.
- Il ramo `if temperature <= 0` — dividere per zero non ha senso, quindi interpretiamo temperature 0 (o negativa) come "prendi sempre e solo il token più probabile": `torch.argmax` restituisce l'indice del logit più alto. Si chiama **greedy decoding** ed è deterministico: stesso input, stesso output, sempre. Utile nei test (vedi 3.6) e per mostrare la differenza fra "scegliere" e "campionare".
- `int(...)` attorno a `.item()` — `.item()` restituisce già un numero Python, il cast esplicito serve solo a rendere felice il type checker dell'IDE.
- `if parola == END: break` — il **criterio di stop**. Se il modello predice il token di fine frase, usciamo dal ciclo prima di aver generato `n_parole` parole: `n_parole` è quindi un *massimo*, non una quantità fissa. Il token `<end>` non finisce mai in `frase`, perché il `break` avviene prima dell'`append`. È il motivo per cui `il gatto dorme` produce `il gatto dorme sul divano` e poi si ferma, invece di continuare con parole a caso.
- Il ciclo `for _ in range(n_parole):` — nota `_` come nome di variabile: convenzione Python per dire "questo valore non mi interessa, mi serve solo ripetere N volte", equivalente a un semplice `for (int i = 0; i < n; i++)` dove non usi mai `i` nel corpo del ciclo.

```
def genera_da_frase(model, stoi, itos, frase_iniziale, n_parole=5, temperature=TEMPERATURE_DEFAULT):
    parole_input = frase_iniziale.strip().split()
    if not parole_input:
        raise ValueError("La frase iniziale è vuota.")

    ultima_parola = parole_input[-1]
    continuazione = generate(model, stoi, itos, ultima_parola, n_parole=n_parole,
                             temperature=temperature)
    parole_generate = continuazione.split()[1:]

    return " ".join(parole_input + parole_generate)
```

`genera_da_frase()` completa una frase intera, ma con un trucco: prende solo **l'ultima parola** (`parole_input[-1]`, indicizzazione negativa: vedi 3.6) come punto di partenza per `generate()`, poi incolla il risultato dietro alla frase originale. `continuazione.split()[1:]` è uno *slice*: "dalla posizione 1 in poi", cioè tutto tranne il primo elemento — scarta `ultima_parola`, che `generate()` include già come prima parola, per non duplicarla.

Il modello non sta "leggendo" l'intera frase: ignora tutto tranne l'ultima parola, perché è un modello *bigram* (una parola di contesto). Il risultato sembra plausibile perché nel nostro piccolo corpus le frasi condividono spesso lo stesso finale, ma è un effetto collaterale, non vera comprensione del contesto lungo. Per un modello che usa davvero tutta la frase come contesto serve l'attention, fuori dallo scope di questo progetto.

```
if __name__ == "__main__":
    argomenti = sys.argv[1:]
    frase_o_parola = " ".join(argomenti) if argomenti else "il"
    n_parole = 5

    if argomenti and argomenti[-1].isdigit():
        n_parole = int(argomenti[-1])
        frase_o_parola = " ".join(argomenti[:-1]) if len(argomenti) > 1 else "il"

    modello, stoi, itos = load_checkpoint()

    if " " in frase_o_parola:
        print(genera_da_frase(modello, stoi, itos, frase_o_parola, n_parole=n_parole))
    else:
        print(generate(modello, stoi, itos, frase_o_parola, n_parole=n_parole))
```

Il blocco eseguibile è un piccolo parser di argomenti a mano. `sys.argv[1:]` sono gli argomenti da riga di comando (args di `main`, senza il nome dello script). `x if cond else y` è l'operatore ternario di Python (`cond ? x : y`), scritto nell'ordine "valore-se-vero, condizione, valore-se-falso". Se l'ultimo argomento è un numero (`.isdigit()`) lo trattiamo come `n_parole`; se resta più di una parola usiamo `genera_da_frase`, altrimenti `generate`. Così funzionano tutte le forme: `python -m src.generate il, ... gatto 8, ... il gatto dorme, ... il gatto dorme 8`.

3.5 demo.py — orchestrazione interattiva

```
def main():
    corpus_corrente = list(CORPUS)
    modello, stoi, itos, _ = train_model(corpus_corrente, verbose=False)

    try:
        while True:
            stampa_menu()
            scelta = input("Scegli un'opzione (1-5): ").strip()

            if scelta == "1":
                ...
            elif scelta == "2":
                frase_grezza = input("Scrivi una nuova frase ...: ").strip()
                nuova_frase = normalizza_testo(frase_grezza)
                if not nuova_frase:
                    print("Frase vuota, ignorata.")
                    continue
                corpus_corrente.append(nuova_frase)
                modello, stoi, itos, storico = train_model(corpus_corrente, verbose=False)
            elif scelta == "3":
                ...
            elif scelta == "5":
                salva = input("Vuoi salvare il modello corrente su disco? (s/n): ").strip().lower()
                if salva == "s":
                    save_checkpoint(modello, stoi, itos)
                break
            else:
                ...
    except (EOFError, KeyboardInterrupt):
        print("\nCiao!")
```

Poca sintassi nuova: un `while True:` (loop infinito, come `while (true)` in Java/C#) con un `if/elif/elif/.../else` (equivalente di `if/else if/.../else`) che chiama le funzioni già spiegate sopra in base alla scelta dell'utente raccolta con `input(...)` (equivalente di leggere da stdin). Alcuni dettagli che valgono una nota:

- `corpus_corrente = list(CORPUS)` — **copia** della lista, non un alias. In Python assegnare una lista a un'altra variabile non la copia (stesso comportamento dei riferimenti in Java); `list(...)` crea una lista nuova. Così le frasi aggiunte durante la demo non toccano mai `CORPUS` originale, e il reset (opzione 4) è sempre affidabile.
- `normalizza_testo(frase_grezza)` — la frase inserita dall'utente viene ripulita (minuscolo, niente punteggiatura, vedi 3.1) prima di entrare nel corpus. `if not nuova_frase:` sfrutta il fatto che una stringa vuota è "falsy" in Python: se dopo la pulizia non resta niente, `continue` torna al menu.
- Ogni volta che il corpus cambia si ri-allena **tutto da zero** (`train_model` ricostruisce vocabolario, dataset e modello): il vocabolario può essere cambiato, quindi la vecchia rete non sarebbe più compatibile. Con 200 epoch su CPU ci vuole meno di un secondo.
- L'opzione 3 usa `genera_da_frase` se il testo contiene uno spazio, altrimenti `generate`, dentro un `try/except ValueError` (vedi 1.13): una parola sconosciuta stampa un messaggio invece di far crashare la demo.
- `except (EOFError, KeyboardInterrupt):` attorno a tutto il loop: Ctrl+D (fine input) o Ctrl+C durante un `input()` fanno uscire con un saluto pulito invece di un traceback poco elegante.
- L'uscita (opzione 5) offre di salvare su disco il modello corrente, così un corpus arricchito durante la demo si può riusare con `python -m src.generate`.

3.6 tests/test_smoke.py — verifica automatica

```
def test_training_riduce_la_loss():
    _, _, _, storico = train_model(epochs=100, verbose=False)
    assert storico[-1] < storico[0]

def test_generate_si_ferma_al_token_di_fine_frase():
    modello, stoi, itos, _ = train_model(epochs=200, verbose=False)
    risultato = generate(modello, stoi, itos, "divano", n_parole=5, temperature=0)
    assert risultato == "divano"
```

- Ogni funzione il cui nome inizia con `test_` viene scoperta ed eseguita automaticamente da **pytest** quando lanci `pytest` da terminale — non serve registrarle da nessuna parte, è una convenzione basata sul nome (come i test JUnit annotati `@Test`, ma qui basta il prefisso nel nome invece di un'annotazione esplicita).
- `assert condizione` — se la condizione è falsa, il test fallisce. Equivalente concettuale di `Assert.IsTrue(...)` / `assertTrue(...)`.
- `storico[-1]` — **indicizzazione negativa**: `-1` significa "l'ultimo elemento della lista", `-2` il penultimo, ecc. Non esiste un equivalente diretto in Java/C# con `[]`; è zucchero sintattico molto comune in Python per evitare `lista[len(lista) - 1]`.
- `_, _, _, storico = train_model(...)` — `train_model` ritorna 4 valori (modello, stoi, itos, storico); qui usiamo `_` per "spacchettare e scartare" i primi tre, tenendo solo il quarto.
- Il secondo test è un buon esempio di come si testa un componente non deterministico: con `temperature=0` (greedy decoding, vedi 3.4) l'output è riproducibile. "divano" compare nel corpus solo a fine frase, quindi il suo unico target di training è `END`: il modello *deve* fermarsi subito e restituire esattamente "divano".

I nove test coprono: forma di vocabolario e dataset, riduzione della loss, output composto solo da parole del vocabolario, stop sul token di fine frase, `<end>` mai presente nell'output, `ValueError` su parola sconosciuta, e i due comportamenti di `genera_da_frase` (prefisso mantenuto; con una sola parola equivale a `generate`). Sono test "di fumo": non verificano la qualità linguistica del modello (con un corpus così piccolo non avrebbe senso), ma che l'intera pipeline funzioni senza errori.

Cose da tenere a mente, in sintesi

Concetto Python	Equivalente concettuale OOP "classico"
Indentazione	{ }
<code>self</code>	<code>this</code> (ma esplicito)
<code>__init__</code>	costruttore
<code>class X(Y):</code>	<code>class X extends Y</code>
Nessun modificatore di accesso reale	tutto <code>public</code> , <code>_nome</code> = convenzione "privato"
Type hints (<code>List[str]</code>)	tipi generici, ma non verificati a compile-time
List/dict comprehension	Stream/LINQ compattati in una riga
Funzione che ritorna una tupla	<code>Tuple<A,B></code> / record multiplo
<code>with ... :</code>	<code>using (...)</code> / try-with-resources
<code>if __name__ == "__main__":</code>	metodo <code>Main</code> opzionale nello stesso file
<code>raise</code> / <code>try</code> / <code>except</code>	<code>throw</code> / <code>try</code> / <code>catch</code> (senza checked exceptions)
Unione di <code>set</code> con l'operatore pipe, <code>{x}</code>	<code>HashSet.addAll</code> / <code>UnionWith</code> , set literal
<code>lista[-1]</code> , <code>lista[1:]</code>	ultimo elemento, sotto-lista dal secondo in poi
<code>x if cond else y</code>	<code>cond ? x : y</code>
<code>model(x)</code> che chiama <code>forward()</code>	override di un metodo di interfaccia invocato dal framework
<code>loss.backward()</code>	differenziazione automatica (autograd): nessun equivalente diretto in OOP classico, è specifico dei framework di deep learning

Riferimenti

- Documentazione ufficiale PyTorch: <https://pytorch.org/docs/stable/index.html>
- PyTorch — nota su `torch.load` e `weights_only`:
<https://pytorch.org/docs/stable/notes/serialization.html>
- Python — guida ufficiale al linguaggio: <https://docs.python.org/3/tutorial/>
- Python — modulo `re` (espressioni regolari): <https://docs.python.org/3/library/re.html>
- pytest — documentazione: <https://docs.pytest.org/>